

Trading Depth for Time in Recurrent Transformers

Zeyi Huang^{12*}, Xuehai He^{1*}, Yong Jae Lee^{2†}, Yelong Shen^{1†}

¹Microsoft

²University of Wisconsin–Madison

Abstract

Recurrent Transformers increase computational depth through temporal recurrence, feeding each token’s high-level hidden state into the computation of the next. This raises a natural question: is additional computation better spent on more temporal steps or greater physical depth? We investigate this question using Latent Recurrent Transformers (LRTs), which retain one backbone forward pass per vocabulary token during decoding and provide a controlled setting for comparing these two ways of adding computation. Specifically, we insert a latent thought token between consecutive vocabulary tokens. Each thought token passes through the same L layers as a vocabulary token, sharing the backbone parameters and providing an additional stage of hidden-state refinement before predicting the next token. We compare this L -layer LRT against a $2L$ -layer LRT without thought tokens. Both execute $2L$ Transformer blocks per vocabulary token during decoding, but the thought-token model uses fewer parameters. On 16- and 20-layer mixture-of-experts NanoChat backbones, one thought token brings the shallower model within 0.006 and 0.004 bits per byte of its double-depth counterpart, recovering 67% and 81% of the improvement with approximately 48% fewer total parameters. These results suggest that temporal thinking offers a parameter-efficient alternative to increasing physical depth in recurrent Transformers.

1 Introduction

A Transformer’s computational depth can be increased in several ways. Increasing physical depth adds independently parameterized layers, allowing each token to undergo more transformations before a prediction. Looped Transformers instead repeatedly apply a shared stack of layers to refine representations at the same token positions [Dehghani et al., 2019, Giannou et al., 2023, Geiping et al., 2026]. When unrolled, an L -layer backbone with R loops executes RL layers per token while retaining the same backbone parameters. Both approaches extend computation along the depth axis, differing primarily in whether successive transformations use distinct or shared parameters.

A different class of recurrent Transformers [Fan et al., 2021, Zeng et al., 2025, Liu and Liu, 2026, Cai et al., 2026, Huang et al., 2026] extends computation along the temporal axis by feeding a high-level hidden state from one token into the computation of the next. This cross-token feedback creates paths that repeatedly traverse the backbone as the sequence unfolds. The next token’s computation can build directly on the preceding token’s completed representation, so a dependency path can continue across vocabulary-token boundaries. This raises a complementary scaling question: rather than increasing the number of layers within each token’s computation, can we obtain similar benefits by introducing additional steps along this recurrent sequence?

We study temporal computation using Latent Recurrent Transformers (LRTs) [Huang et al., 2026], which reuse the preceding token’s high-level hidden state through hidden-state and KV feedback. During autoregressive decoding, that state is already available, allowing LRT to retain one backbone forward pass per vocabulary token. LRT therefore provides a natural starting point for studying how to allocate additional computation: we can introduce more temporal steps or increase physical depth while keeping the recurrent architecture fixed.

Specifically, we insert a latent thought token between consecutive vocabulary tokens, providing another stage of hidden-state refinement before predicting the next token. The thought token passes through the same backbone and shares its parameters with the vocabulary-token computation. Its final state then supplies

Work mainly done during Zeyi’s internship at Microsoft. *† denote equal contribution and advising.

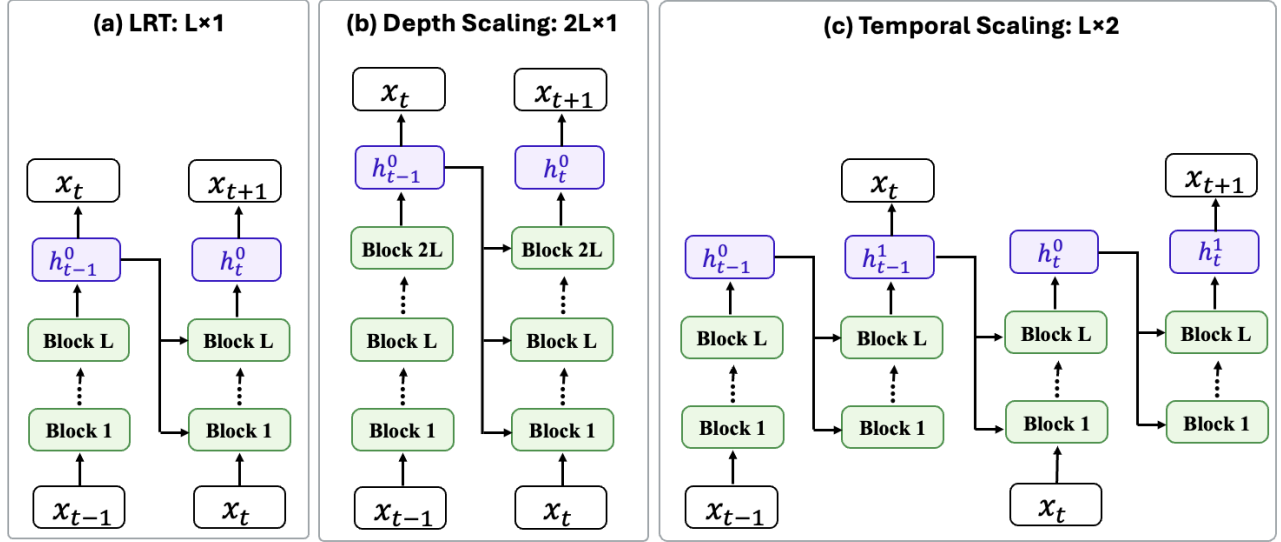


Figure 1: Scaling LRT along depth or time. (a) An L -layer LRT performs one backbone forward pass per vocabulary token and feeds its final hidden state into the next token’s computation. (b) Depth scaling increases the backbone to $2L$ independently parameterized layers. (c) Temporal scaling inserts one thought token after each vocabulary token, reusing the same L -layer backbone. The vocabulary-token state $h_t^{(0)}$ is refined into the thought state $h_t^{(1)}$, which predicts x_{t+1} and supplies feedback to its computation, continuing the recurrent chain across token boundaries.

feedback to the next vocabulary token, preserving the cross-token recurrent chain. We compare this L -layer LRT with one thought token against a $2L$ -layer LRT without thought tokens. Both execute $2L$ blocks per vocabulary token during decoding, allowing us to ask whether reusing existing layers along time can recover the benefits of adding distinct layers along depth. This comparison matches the number of block executions, while attention overhead and training procedures also affect actual cost.

On 16- and 20-layer NanoChat MoE backbones, one thought token recovers 67% and 81% of the BPB improvement from doubling LRT depth, using approximately 48% fewer total parameters. A second thought token further improves performance at both model sizes, although triple-depth LRTs remain better at the same decoding block count. We also compare with looped Transformers as a baseline for extending computation along the depth axis, and with PonderLM-2 [Zeng et al., 2025] along the temporal axis. Unlike PonderLM-2, LRT with thought tokens feeds the final thought state into the next vocabulary token’s computation, extending the recurrent dependency chain across token boundaries. Removing this connection largely closes the performance gap with PonderLM-2, supporting the longer recurrent chain as a key contributor to the gains of LRT with thought tokens. Together, these experiments examine both the depth–time trade-off and the feedback connections that support temporal computation.

2 Method

Starting from an L -layer Latent Recurrent Transformer (LRT), we study two ways to increase computation: adding independently parameterized layers to the backbone, or inserting latent thought tokens that reuse its existing layers. Both retain LRT’s feedback across vocabulary tokens, so the central comparison changes how additional computation is allocated within the same recurrent architecture. We first describe the backbone and thought-token construction, then explain the depth–time comparison and the parallel training procedure. Figure 1 illustrates the two scaling choices.

2.1 Latent Recurrent Backbone

Let $x_1, \dots, x_N$ be a sequence of vocabulary tokens, and let $E[x_t]$ denote the embedding of token x_t . LRT passes the preceding token’s top-layer hidden state into the current token’s computation. Using the final layer as the

recurrent source, an L -layer LRT computes

$$h_t = F_\theta(E[x_t], h_{t-1}; C_{<t}), \quad (1)$$

where F_θ denotes the backbone together with its feedback pathways, h_{t-1} is the preceding token’s final hidden state, and $C_{<t}$ contains the earlier positions’ KV states. Each forward pass also adds the current position’s KV states to the cache; we omit those updates from the equation for readability. The model predicts x_{t+1} from h_t using its output head.

This feedback allows the current prediction to build on a representation that has already passed through the backbone at the preceding token. Since h_{t-1} is available during autoregressive decoding, the recurrence requires only one new backbone forward pass per vocabulary token.

Feedback configuration. We follow the LRT configuration with one change: the recurrent source is the final-layer hidden state, rather than a searched intermediate layer, which we leave to future work. The state enters the current computation in two ways: through hidden-state residuals and through projected keys and values. Learned scales and gates control these contributions. We keep this feedback design fixed when varying physical depth or adding thought tokens, so the scaling comparison uses the same recurrent mechanism.

2.2 Adding Thought Tokens

We add temporal computation by inserting a continuous thought position after each vocabulary token. Unlike a vocabulary position, a thought position does not correspond to a discrete token from the vocabulary. It takes the preceding hidden state as its input and provides another backbone forward pass before the next vocabulary token is predicted.

Consider one thought token. The model first processes the current vocabulary token, then runs its resulting hidden state through the same backbone once more. It predicts the next vocabulary token from this thought state and passes the state forward through LRT’s recurrent pathway. Thus, computation follows alternating vocabulary and thought positions, with feedback both from the vocabulary position to its thought position and from that thought position to the next vocabulary position. No vocabulary token is sampled while processing the thought token.

More generally, let $h_t^{(0)}$ be the state after processing x_t , and $h_t^{(j)}$ the state after processing its j th thought token. With K thought tokens per vocabulary token,

$$h_t^{(0)} = F_\theta(E[x_t], h_{t-1}^{(K)}; C_{<t,0}), \quad (2)$$

$$h_t^{(j)} = F_\theta(h_t^{(j-1)}, h_t^{(j-1)}; C_{<t,j}), \quad j = 1, \dots, K. \quad (3)$$

Here $C_{<t,j}$ contains all positions preceding the current step. The two occurrences of $h_t^{(j-1)}$ in Eq. (3) have different roles: the state replaces a token embedding at the input and also supplies the recurrent feedback within the backbone. The original output head reads the final state $h_t^{(K)}$ to predict x_{t+1} . Setting $K = 0$ recovers LRT without thought tokens.

Parameter sharing and attention. Thought tokens have no learned token embedding. Vocabulary and thought positions share the attention, MoE, and feedback-projection parameters; only their feedback scales and gates are separate. Adding thought tokens therefore introduces no additional independently parameterized backbone layers. We use an MoE backbone so that the router can learn to select different experts for vocabulary tokens and for thought tokens at different thinking steps. This allows successive forward passes to activate different subsets of the shared parameters as the hidden state evolves. All positions write KV states and remain visible to later positions under the attention mask, allowing subsequent computation to attend to earlier thought representations as well as vocabulary-token representations. Thought tokens share their associated vocabulary token’s rotary position index: for $K = 1$, the indices are $0, 0, 1, 1, \dots$, and for $K = 2$, they are $0, 0, 0, 0, 1, 1, 1, 1, \dots$.

Cross-token feedback. The key distinction from PonderLM-2 is how computation continues after a thought token. LRT with thought tokens feeds the final thought state directly into the next vocabulary token’s

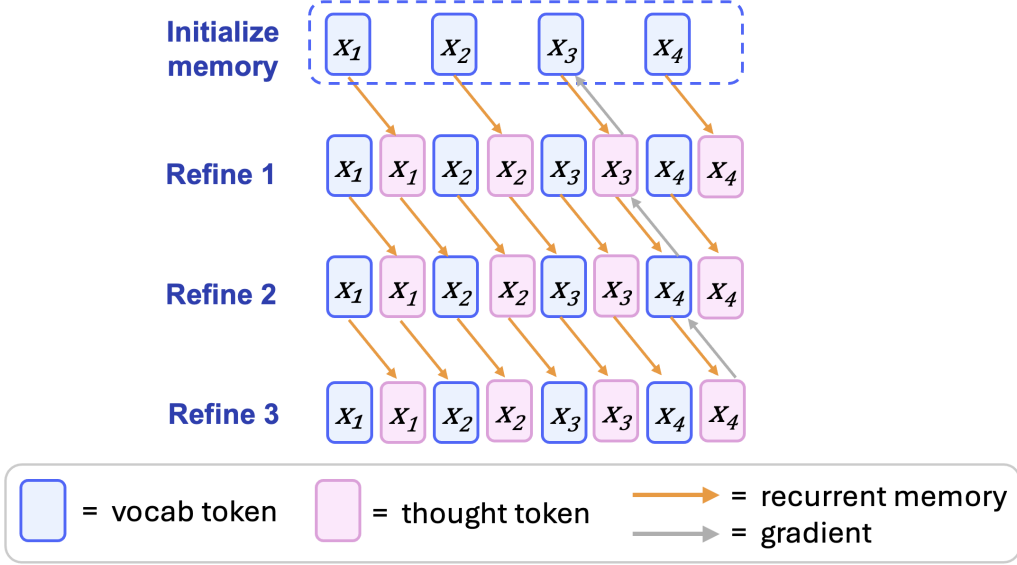


Figure 2: Parallel multi refinement training and gradient flow. A vocabulary-only forward pass initializes recurrent memory, followed here by three refinement passes over vocabulary (blue) and thought (pink) positions. Each box represents one forward pass through the shared L -layer backbone. Orange arrows pass final hidden states from the preceding pass to the next position in the expanded sequence. For clarity, gray arrows show only the recurrent gradient path from the thought token associated with x_4 in Refine 3. This path runs backward through vocabulary token x_4 in Refine 2, thought token x_3 in Refine 1, and vocabulary token x_3 in initialization. It connects four backbone passes across two vocabulary-token intervals, illustrating how recurrent dependencies accumulate approximately $4L$ computational depth across refinement passes. Attention connections and other gradient paths are omitted.

computation, allowing the recurrent dependency chain to continue across vocabulary-token boundaries. Without this connection, the explicit feedback chain ends at the thought token, although information remains accessible through causal attention. Section 3.5 examines the contribution of this cross-token connection.

2.3 Comparing Physical Depth and Temporal Computation

We hold width and the recurrent feedback design fixed, and compare adding independently parameterized layers with adding thought tokens that reuse the existing layers. An L -layer model with K thought tokens executes

$$B_{\text{decode}}(L, K) = L(K + 1) \quad (4)$$

blocks per vocabulary token during decoding. Thus, one thought token matches the decoding block count of doubling depth, and two match that of tripling depth. The deeper model applies additional distinct layers, whereas the temporal model repeatedly applies the shared backbone. For looped models, the decoding block count is the physical layer count multiplied by the number of loops.

Parameters and attention cost. Thought tokens add only a negligible number of feedback scales and gates, while depth scaling adds full backbone layers. Doubling physical depth need not exactly double total parameters, since token embeddings and the output projection do not scale with the number of layers.

Equal block counts also do not imply equal FLOPs or latency. With full causal attention and equal KV dimensions, a $2L$ -layer model over N vocabulary positions and an L -layer model over $2N$ vocabulary-and-thought positions both store $2LN$ layer–position KV entries. Their leading-order KV storage is therefore comparable, but the thought-token model attends over more positions and incurs greater attention interaction cost. Sliding windows and MoE routing also affect actual cost. Our decoding comparison uses block executions to describe the allocation of computation; serving efficiency additionally requires measured latency and throughput.

2.4 Parallel Multi Refinement Training

The equations above describe sequential decoding, where each completed state is immediately available to the next position. Applying the same recurrence exactly during training would require each position to wait for the preceding position’s final state, preventing a straightforward parallel forward over the sequence. We instead use multi refinement training [Zeng et al., 2025, Huang et al., 2026]: each pass processes all positions in parallel, taking recurrent inputs from the preceding pass’s final states shifted by one position. For thought-token models, these positions belong to the expanded vocabulary-and-thought sequence. At thought positions, the shifted state also supplies the input embedding replacement. Figure 2 illustrates this procedure.

Training procedure. The LRT thought-token models begin with one vocabulary-only initialization pass with zero recurrent input and no direct prediction loss. This pass provides the initial states for four refinement passes over $N(K + 1)$ positions. LRTs without thought tokens run four passes over vocabulary positions.

We apply next-token loss in each supervised pass with equal weight and retain gradients across passes. In LRT with thought tokens, the prediction is read from the last thought state of each vocabulary-token interval, so the vocabulary position itself receives no direct prediction loss. PonderLM-2 uses a similar multi refinement training procedure, with more refinement passes on average (4.5). Looped Transformers instead unroll their repeated backbone forward passes in the training graph. The overall training-budget comparison is described in Section 3.1.

3 Experiments

3.1 Setup

We evaluate NanoChat MoE models at two reference sizes, with depths $L_0 = 16$ and $L_0 = 20$, following the base experimental setup of [Huang et al., 2026]. Depth multipliers of $1\times$, $2\times$, and $3\times$ correspond to 16/32/48 layers and 20/40/60 layers. For each reference model size, width is fixed across depth-scaled variants. The MoE configuration uses eight routed experts, top-2 routing, and one shared expert. Thought-token and looped models retain the reference physical depth.

Model sizes. The reference 16-layer and 20-layer vanilla Transformers contain 0.8B and 1.3B total parameters, respectively. Parameter multipliers are normalized separately to these two reference models and include all MoE experts. The total parameter count of each other configuration is obtained by multiplying the relevant reference count by its ratio in Table 1. LRT adds approximately 4% and 5% to the reference models, while thought tokens reuse the same backbone and leave the reported total-parameter ratios unchanged.

Evaluation. We report validation bits per byte (BPB; lower is better), averaged over three seeds. Evaluation starts from the first vocabulary token and executes the sequential recurrence with KV caching. At each step, the model receives the ground-truth token and scores the next ground-truth token. Models with one and two thought tokens are trained separately, with the same number of thought tokens used during training and evaluation.

Training budgets. The LRT, PonderLM-2, and looped configurations use broadly comparable training FLOP budgets in the reported runs. LRT and PonderLM-2 incur repeated computation through multi refinement training, while looped models execute multiple backbone forward passes. Standard Transformers do not use either mechanism and are not FLOP-matched to these models; their results provide a reference for physical-depth scaling. The budget comparison among the recurrent and looped models is approximate, rather than an exact per-configuration FLOP match. Table 1 reports decoding block counts separately from training cost. .

3.2 One Thought Token Recovers Much of the Gain from Doubling Depth

At the 16-layer reference size, doubling LRT depth improves BPB from 0.798 to 0.780, while adding one thought token reaches 0.786. At the 20-layer reference size, the corresponding values are 0.762, 0.741, and 0.745. One

Table 1: Depth versus temporal computation. BPB is averaged over three seeds. Physical depth and total parameters are normalized to the corresponding 16-layer or 20-layer reference Transformer. Decode gives the relative number of Transformer-block executions per vocabulary token during decoding, accounting for both physical depth and repeated backbone forward passes. It excludes additional attention overhead and does not measure training FLOPs or latency. Within each Decode group containing multiple models, bold and underlined BPB values indicate the best and second-best results, respectively, separately for each reference model size.

Model	Physical Depth	Decode	Total params		BPB ↓	
			16L	20L	16L	20L
Transformer	1×	1×	1.00×	1.00×	<u>0.818</u>	<u>0.788</u>
Transformer	2×	2×	1.92×	1.94×	0.798	0.767
Transformer	3×	3×	2.83×	2.87×	0.789	0.756
Loop Transformer, 2 loops	1×	2×	1.00×	1.00×	0.804	0.774
Loop Transformer, 3 loops	1×	3×	1.00×	1.00×	0.799	0.768
Loop Transformer, 4 loops	1×	4×	1.00×	1.00×	0.795	0.764
PonderLM-2, 1 thought token	1×	2×	1.00×	1.00×	0.797	0.752
PonderLM-2, 2 thought tokens	1×	3×	1.00×	1.00×	0.789	0.745
LRT	1×	1×	1.04×	1.05×	0.798	0.762
LRT	2×	2×	2.00×	2.04×	0.780	0.741
LRT	3×	3×	2.96×	3.03×	0.771	0.731
LRT+ 1 thought token	1×	2×	1.04×	1.05×	<u>0.786</u>	<u>0.745</u>
LRT+ 2 thought tokens	1×	3×	1.04×	1.05×	<u>0.779</u>	<u>0.739</u>

thought token therefore recovers 66.7% and 81.0% of the BPB reduction from doubling depth, leaving gaps of 0.006 and 0.004 BPB.

The thought-token models use 48.0% and approximately 48.5% fewer total parameters than their double-depth counterparts, calculated from Table 1’s rounded parameter ratios. Both alternatives execute twice the reference number of blocks per vocabulary token during decoding. Thus, reusing the backbone recovers much of the quality gain from extra layers, although additional distinct layers remain better at both model sizes.

3.3 Further Gains from Additional Thought Tokens

Increasing the number of thought tokens from one to two improves BPB from 0.786 to 0.779 at the 16-layer reference size and from 0.745 to 0.739 at the 20-layer reference size. These models use essentially the same parameters as their counterparts with one thought token, but execute three backbone forward passes per vocabulary token.

At the same decoding block count, triple-depth LRTs reach 0.771 and 0.731 BPB. LRTs with two thought tokens come within 0.008 BPB of these models at both model sizes, recovering 70.4% and 74.2% of the improvement from tripling depth while using approximately 65% fewer total parameters. These results extend the depth–time trade-off beyond a single thought token: additional temporal steps further improve performance and recover a substantial fraction of the gains from greater physical depth.

3.4 Comparison with Other Architectures

Looped Transformers. At two backbone forward passes per vocabulary token, LRT with one thought token improves over the two-loop Transformer by 0.018 BPB at the 16-layer reference size and 0.029 at the 20-layer reference size. With two thought tokens, the gaps over three-loop Transformers are 0.020 and 0.029. LRT with one thought token also outperforms the four-loop models at both model sizes. The LRT variants have a small total-parameter overhead of 4–5% relative to the looped models.

Table 2: Feedback ablations at depth 16 with one thought token. Each ablation independently removes one component from the full LRT configuration. PonderLM-2 is included as a reference without explicit cross-token recurrent feedback.

Configuration	BPB ↓	Increase over full LRT
LRT+ 1 thought token	0.786	—
Without cross-token recurrence	0.795	+0.009
Without KV feedback	0.790	+0.004
PonderLM-2, 1 thought token	0.797	+0.011

PonderLM-2. With one thought token, LRT reaches 0.786 versus 0.797 BPB at the 16-layer reference size and 0.745 versus 0.752 at the 20-layer reference size. With two thought tokens, the corresponding comparisons are 0.779 versus 0.789 and 0.739 versus 0.745. LRT has lower BPB at both thought-token counts and both scales. The feedback ablations below examine two architectural differences within the LRT configuration.

Standard Transformers. LRT with one thought token also has lower BPB than a triple-depth standard Transformer: 0.786 versus 0.789 and 0.745 versus 0.756. It uses fewer total parameters and fewer decoding block executions. Because the standard Transformer runs use less training compute, these results describe parameter and decoding trade-offs rather than a training-FLOP-matched advantage.

3.5 Ablating Cross-Token Recurrence and KV Feedback

We ablate the 16-layer LRT with one thought token to examine two aspects of its feedback design (Table 2).

Cross-token recurrence. PonderLM-2 introduces thought tokens but does not explicitly feed the final thought state into the next vocabulary token’s computation. LRT with thought tokens includes this connection, extending the recurrent dependency chain across vocabulary-token boundaries. Removing it preserves the thought-token computation and causal attention, but breaks the explicit recurrent chain between successive token intervals. BPB increases from 0.786 to 0.795, approaching PonderLM-2’s 0.797. This ablation largely closes the performance gap with PonderLM-2, supporting the longer recurrent dependency chain as a key contributor to the gains of LRT with thought tokens.

KV feedback. In a separate ablation, we remove KV feedback while retaining hidden-state feedback and cross-token recurrence. BPB increases from 0.786 to 0.790, indicating that KV feedback provides an additional benefit beyond hidden-state feedback alone.

4 Discussion and Limitations

Shared temporal computation offers a useful parameter–quality trade-off: one or two thought tokens recover a substantial fraction of the gains from doubling or tripling depth, but deeper LRTs remain better at the same decoding block count. This pattern is consistent with additional independent layers providing capacity that repeated shared transformations do not fully replace; the present experiments do not isolate the cause of the remaining gap.

Training budgets among recurrent and looped runs are approximately comparable; per-configuration measured FLOPs would make this comparison more precise. Decoding block counts omit attention overhead and are not latency or throughput measurements. Finally, models with different numbers of thought tokens are trained separately; generalization to more thought tokens or adaptive thought-token counts at inference remains an open question.

5 Conclusion

Adding a shared thought token to an LRT recovers 67% and 81% of the BPB improvement from doubling physical depth, with approximately 48% fewer total parameters. A second thought token improves performance at both model sizes, while triple-depth LRTs retain an advantage at the same decoding block count. Feedback ablations support the contribution of cross-token recurrence and KV feedback. Temporal computation is therefore a useful way to recover much of the benefit of additional layers while keeping the backbone compact.

References

- Ziyang Cai, Xingyu Zhu, Yihe Dong, Yinghui He, and Sanjeev Arora. T²mlr: Transformer with temporal middle-layer recurrence. *arXiv preprint arXiv:2607.15178*, 2026.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.
- Angela Fan, Thibaut Lavril, Edouard Grave, Armand Joulin, and Sainbayar Sukhbaatar. Addressing some limitations of transformers with feedback memory. *arXiv preprint arXiv:2002.09402*, 2021.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *Advances in Neural Information Processing Systems*, 38:41340–41391, 2026.
- Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 11398–11442, 2023.
- Zeyi Huang, Xuehai He, LiLiang Ren, Yiping Wang, Baolin Peng, Hao Cheng, Shuohang Wang, Pengcheng He, Jianfeng Gao, Yong Jae Lee, et al. Latent recurrent transformer: Architecture exploration, training strategies, and scaling behavior. *arXiv preprint arXiv:2605.26797*, 2026.
- Bo Liu and Qiang Liu. Maglev: Sliding recurrent memory. *arXiv preprint arXiv:2608.02870*, 2026.
- Boyi Zeng, He Li, Shixiang Song, Yixuan Wang, Zitong Wang, Ziwei He, Xinbing Wang, and Zhouhan Lin. Ponderlm-2: Pretraining llm with latent thoughts in continuous space. *arXiv preprint arXiv:2509.23184*, 2025.